LLM-Based Automated Python Test Generation and Mutation Testing Pipeline

Problem Statement

Design and implement an LLM-based automated testing pipeline for Python programs that generates, evaluates, and iteratively improves unit tests using mutation feedback.

You are required to build a framework that uses a Large Language Model (LLM) to automatically generate unit tests for a given Python program. The framework must execute the generated tests using **pytest** and evaluate their effectiveness using mutation testing tools such as **mutmut**, **Cosmic Ray**, or **mutpy**.

The following GitHub repository should be used for creating the automated testing pipeline:

https://github.com/pytransitions/transitions

Objectives and Requirements

1. **Initial Test Generation**

   Develop an LLM-based testing framework capable of automatically generating **pytest** unit tests for a given Python program.

   The generated test suite should provide coverage for:

   - Normal cases
   - Edge cases
   - Invalid inputs

2. **Test Execution and Metrics**

   The framework should automatically execute the generated tests using **pytest** and compute the following evaluation metrics:

   - Code coverage using **coverage.py** (or an equivalent tool)
   - Mutation score using **mutmut**, **Cosmic Ray**, or **mutpy**

3. **Mutation Analysis**

   The system should perform mutation analysis by:

   - Identifying surviving mutants after mutation testing.
   - Extracting and representing surviving mutants in a structured format (for example, operator changes, removed conditions, etc.).

4. **Augmented Prompting**

   Design an adaptive prompting mechanism that:

   - Takes surviving mutants as input.
   - Analyzes weaknesses in the current test suite.
   - Generates improved or additional test cases.

   The prompt provided to the LLM must explicitly include:

   - Surviving mutants.
   - Current coverage percentage.
   - Current mutation score.

5. **Iterative Feedback Loop**

   Implement an iterative feedback pipeline following the workflow:

   Generate Tests → Execute → Mutate → Evaluate → Augment Prompt → Improve Tests → Repeat

   The framework should execute multiple iterations and terminate when either:

   - The mutation score exceeds a predefined threshold (for example, 90%), or
   - The maximum number of iterations has been reached.

6. **Metrics Tracking**

   At every iteration, record the following metrics:

   - Coverage (%)
   - Mutation Score (%)
   - Number of surviving mutants

   The results should be displayed in a tabular or logged format.

7. **Pipeline Design**

   Implement the framework as either:

   - A modular pipeline, or
   - A multi-agent system (optional but encouraged).

   Example agents include:

   - Test Generator Agent
   - Mutation Analyzer Agent
   - Prompt Refiner Agent

- Execution Agent

Output Requirements

The completed system should generate the following outputs:

1. Generated test cases (initial and improved).
2. Test execution results.
3. Mutation testing results.
4. Iteration-wise metrics table.
5. Final mutation score and code coverage.
6. Logs or explanations describing the improvements made during each iteration.